

Encontro 01: Containers

Containerização com Docker



INTELI

Engenharia de Computação
Módulo 07 - Sistema de manutenção
preditiva com IA e arquitetura em nuvem

Agosto - 2026

Encontro 01: containers e containerização

Objetivo do encontro:

- Entender qual problema a containerização resolve
- Diferenciar máquina virtual de container pelo que cada um virtualiza
- Compreender as três bases no kernel: namespaces, cgroups e sistema de arquivos em camadas
- Aplicar limites de recursos com cgroups através do Docker
- Executar os primeiros containers e ler o que está acontecendo na máquina

Por que os containers existem

O problema nunca foi empacotar código. Foi reproduzir o ambiente.

0 problema que aparece antes da tecnologia

- Máquinas diferentes: sistema operacional, versões de bibliotecas e configurações que não coincidem
- Aplicações diferentes na mesma máquina, exigindo versões conflitantes da mesma dependência
- Necessidade de isolamento: um serviço com problema não pode derrubar os demais
- Necessidade de escala: subir e derrubar réplicas rapidamente, sem preparar uma máquina inteira
- Custo: uma máquina por aplicação desperdiça recurso ocioso

"Na minha máquina funciona" é a descrição informal de um problema real: o ambiente de execução não faz parte da entrega.

Três respostas para o mesmo problema

Servidor físico

- Uma aplicação por máquina
- Isolamento total
- Provisionamento lento e recurso ocioso

Máquina virtual

- Hipervisor virtualiza o hardware
- Cada VM traz o seu próprio sistema operacional
- Isolamento forte, custo de memória, disco e tempo de boot

Container

- Virtualiza o sistema operacional, não o hardware
- Compartilha o kernel do host
- Processo isolado: inicia em segundos e ocupa pouco

Máquina virtual e container: o que muda

Máquina virtual

- Camada: hardware, hipervisor, sistema operacional convidado, aplicação
- Cada VM roda o seu próprio kernel
- Inicialização em dezenas de segundos ou minutos
- Imagem medida em gigabytes

Container

- Camada: hardware, sistema operacional do host, engine, container
- Todos os containers usam o kernel do host
- Inicialização em milissegundos ou segundos
- Imagem medida em megabytes, com camadas compartilhadas

A pergunta que separa os dois: o que está sendo virtualizado, o hardware ou o sistema operacional.

Container não substitui máquina virtual em tudo

- Kernel diferente do host: rodar Windows sobre Linux, ou um kernel com versão específica, exige virtualização
- Isolamento de segurança forte entre cargas que não confiam umas nas outras
- Cargas que dependem de acesso direto a hardware ou de módulos de kernel próprios
- Na prática o arranjo comum é combinado: containers executando dentro de máquinas virtuais na nuvem

No Windows e no macOS o Docker roda dentro de uma máquina virtual Linux. O kernel compartilhado é o dessa VM, não o do sistema hospedeiro.

O que é um container, de verdade

Não é uma máquina pequena. É um processo do host com a visão restrita.

Um container é um processo com restrições

Analogia: o container é uma cesta de piquenique. Tudo o que a aplicação precisa para executar vai dentro dela.

Definição técnica: é um ou mais processos executando no kernel do host, com a visão do sistema restrita por namespaces, com o consumo de recursos limitado por cgroups e com o sistema de arquivos montado a partir de camadas de imagem.

Verificação prática: o processo do container aparece no ps do host. Não existe uma máquina nova, existe um processo com regras diferentes.

Sem essas três peças não existe container. O Docker é a ferramenta que as configura de forma conveniente.

As três bases da containerização

Namespaces

- Respondem: o que o processo enxerga
- Processos, rede, montagens, hostname e usuários
- Recurso do kernel Linux

cgroups

- Respondem: quanto o processo pode consumir
- CPU, memória, número de processos e I/O
- Também contabilizam o uso

Camadas de imagem

- Respondem: de onde vem o sistema de arquivos
- Union filesystem, normalmente overlayfs
- Camadas somente leitura mais uma camada de escrita

Namespaces isolam a visão. cgroups isolam o consumo. As camadas entregam o disco.

Namespaces: o que cada um isola

Cada namespace recorta uma parte da visão que o processo tem do sistema.

Namespace	O que ele isola
<code>pid</code>	A árvore de processos. O processo principal do container é o PID 1 dentro dele
<code>mnt</code>	Os pontos de montagem, ou seja, qual sistema de arquivos o processo enxerga
<code>net</code>	Interfaces, endereços IP, portas e tabelas de roteamento
<code>uts</code>	O hostname e o nome de domínio da máquina
<code>ipc</code>	Memória compartilhada e filas de mensagens entre processos
<code>user</code>	O mapeamento de usuários e grupos, permitindo root dentro sem root fora
<code>cgroup</code>	A visão da hierarquia de cgroups a que o processo pertence
<code>time</code>	Os relógios monotônico e de boot, disponível em kernels recentes

Demonstração 1: enxergando os namespaces

```
# terminal 1: sobe um container interativo
docker run -it --rm --name lab alpine sh
# dentro do container:
hostname          # outro nome de máquina (namespace uts)
ps aux            # só os processos do container (namespace pid)
ip addr           # outra interface de rede (namespace net)
```

```
# terminal 2: no host, o mesmo processo existe
docker inspect -f "{{.State.Pid}}" lab
ps -p <PID> -o pid,ppid,cmd
sudo ls -l /proc/<PID>/ns
```

Ponto de discussão: o processo tem um PID no host e outro dentro do container. É a mesma execução, vista de dois lugares.

cgroups: control groups

Namespaces dizem o que o processo vê. cgroups dizem quanto ele pode usar.

cgroups: o que são e por que existem

- **Definição:** recurso do kernel Linux que agrupa processos e aplica limite, contabilização e priorização de recursos a esse grupo
- **Sem cgroups:** um único container poderia consumir toda a memória ou toda a CPU e derrubar os vizinhos, mesmo estando isolado por namespaces
- **Organização:** os grupos formam uma hierarquia, e os limites de um grupo valem para todos os processos abaixo dele
- **Versões:** a v1 usa uma hierarquia por controlador, a v2 usa hierarquia única e é o padrão nas distribuições atuais
- **Quem configura:** no dia a dia o systemd e o runtime (runc) criam os grupos, e o Docker escreve os limites que você pediu na linha de comando

Interface: os grupos são arquivos em `/sys/fs/cgroup`. Ler e escrever nesses arquivos é o mecanismo, o resto é conveniência.

Os controladores que importam no dia a dia

O que cada um controla

- cpu: cota e peso de processamento
- memory: limite de memória do grupo
- pids: número máximo de processos e threads
- io: banda e operações por segundo em disco
- cpuset: em quais núcleos o grupo pode executar

O que acontece ao atingir o limite

- memory: o kernel tenta recuperar memória e, se não conseguir, encerra o processo (OOM kill, saída 137)
- cpu: não encerra nada, aplica throttling. O efeito aparece como lentidão, não como erro
- pids: novas criações de processo falham, o que contém uma fork bomb
- io: as operações passam a esperar

Memória é limite rígido, CPU é limite elástico. Confundir os dois é a origem de boa parte dos diagnósticos errados em produção.

Do comando do Docker ao arquivo do cgroup

A flag da linha de comando termina como um valor escrito em um arquivo do kernel.

Flag do docker run	Arquivo (cgroup v2)	Efeito
<code>--memory=128m</code>	<code>memory.max</code>	Limite de memória. Ao estourar, o processo é encerrado (OOM kill, saída 137)
<code>--memory-reservation=64m</code>	<code>memory.low</code>	Limite flexível, respeitado quando o host está sob pressão
<code>--cpus=0.5</code>	<code>cpu.max</code>	Cota de tempo por período. Ao estourar, aplica throttling
<code>--cpu-shares=512</code>	<code>cpu.weight</code>	Peso relativo na disputa por CPU, sem teto fixo
<code>--cpuset-cpus=0,1</code>	<code>cpuset.cpus</code>	Fixa em quais núcleos o container pode executar
<code>--pids-limit=50</code>	<code>pids.max</code>	Número máximo de processos e threads no container

Demonstração 2: cgroups na prática

```
# sobe um container com limites e observa o consumo
docker run -d --name limites --memory=128m --memory-swap=128m \
  --cpus=0.5 --pids-limit=50 nginx:alpine
docker stats limites
docker inspect -f "{{.Id}}" limites
```

```
# no host, lendo o limite direto do kernel (cgroup v2)
cat /sys/fs/cgroup/system.slice/docker-<ID>.scope/memory.max
cat /sys/fs/cgroup/system.slice/docker-<ID>.scope/cpu.max
```

```
# forçando o limite de memória: o container é encerrado
docker run --rm --memory=64m --memory-swap=64m alpine \
  sh -c "dd if=/dev/zero of=/dev/shm/bloco bs=1M count=200"
```

Se o driver de cgroup for cgroupfs, o caminho é `/sys/fs/cgroup/docker/<ID>/`. Verifique com `docker info | grep -i cgroup`.

Por que os cgroups importam para o projeto de vocês

- Vizinho barulhento: sem limite, um container em laço infinito degrada todos os outros na mesma máquina
- Previsibilidade: com limite declarado, o comportamento sob carga é o mesmo na máquina do time e no servidor
- Densidade e custo: limites permitem colocar mais serviços na mesma máquina com segurança
- Diagnóstico: saída 137 indica encerramento por falta de memória, e lentidão sem erro costuma ser throttling de CPU
- Base para o próximo passo: requests e limits do Kubernetes são exatamente cgroups configurados por outra camada

Um container sem limites é um processo comum do host com um nome bonito.

0 que o container não isola

- O kernel é compartilhado: uma vulnerabilidade no kernel do host alcança todos os containers
- Sem user namespace ativado, o root dentro do container é o root do host em caso de escape
- A flag --privileged desliga boa parte das proteções, e por isso não é padrão de uso
- Por padrão, o processo dentro do container ainda lê /proc/meminfo do host: ferramentas antigas ignoram o limite do cgroup
- Container não é fronteira de segurança equivalente a uma máquina virtual

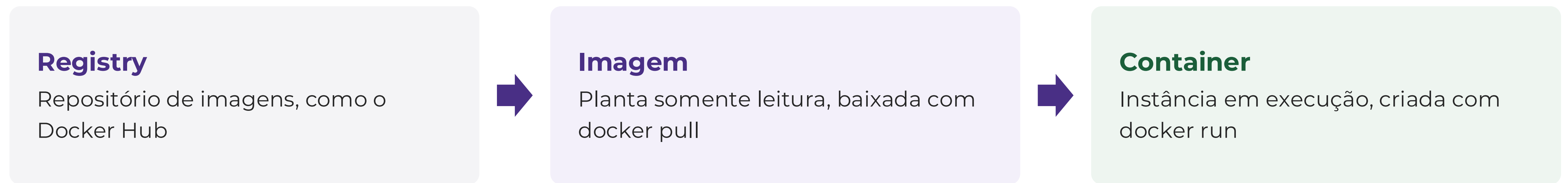
Isolamento é um conjunto de escolhas de configuração, e não uma propriedade automática de quem usa containers.

Docker na prática

Engine, imagem, container e registry.

Os elementos básicos

O container engine é o que traduz o seu comando em namespaces, cgroups e montagens.



```
docker run hello-world      # valida a instalação
docker version              # cliente e engine
docker info | grep -i cgroup
```

A imagem é uma planta feita de camadas

- A imagem está para o container como a classe está para o objeto: uma descreve, a outra executa
- A imagem é um conjunto de camadas somente leitura, empilhadas por um sistema de arquivos union, normalmente o overlayfs
- Ao lançar o container, o engine adiciona uma camada de escrita no topo, com cópia sob escrita
- As camadas são compartilhadas entre imagens e containers, o que explica o download rápido e o pouco espaço em disco
- Tudo que é escrito na camada superior desaparece quando o container é removido, e é isso que os volumes resolvem

Cada docker run cria um container novo a partir da imagem. Para voltar a um container existente use docker start.

Primeiros comandos e os dois modos de execução

```
docker pull nginx:alpine      # baixa a imagem
docker images                 # imagens locais
docker run -d --name web -p 8080:80 nginx:alpine # modo detached
docker run -it --rm alpine sh  # modo attached, terminal interativo
docker ps                    /   docker ps -a
docker logs web              /   docker exec -it web sh
docker stop web              /   docker start web      /   docker rm -f web
```

Attached

- O terminal fica preso ao container
- Você recebe a saída padrão na tela
- Encerrar o terminal encerra a execução

Detached

- O container roda em segundo plano
- O terminal permanece livre
- A saída é lida com docker logs

Demonstração 3: um serviço com limites

```
docker run -d --name web -p 8080:80 \
  --memory=128m --cpus=0.5 nginx:alpine
curl http://localhost:8080
docker stats web
docker exec -it web sh -c "cat /proc/1/cgroup"
```

- Publicar a porta com -p 8080:80 conecta o namespace de rede do container ao host
- O mesmo serviço, com e sem limite, permite comparar o comportamento sob carga
- Encerrar com `docker rm -f web` e observar que a camada de escrita foi descartada

Fechamento do ciclo: imagem baixada, container criado, recursos limitados, porta publicada e execução observada.

Resumo e próximos passos

- Container é um processo do host com visão restrita por namespaces e consumo limitado por cgroups
- A máquina virtual virtualiza o hardware, o container virtualiza o sistema operacional e compartilha o kernel
- A imagem é a planta em camadas somente leitura, e o container acrescenta a camada de escrita
- Limite de memória encerra o processo, limite de CPU aplica throttling
- Isolamento depende de configuração: kernel compartilhado, usuário e privilégios precisam de atenção

Próximo encontro: Dockerfile para construir a própria imagem, volumes para persistir dados e composição de serviços.



Murilo Zanini de Carvalho

murilo.zanini@prof.inteli.edu.br

